- Tanzania Climate Prediction Part 2: Bridging the Valley of Death from Prototype to Production MLOps
  - 📌 Project Evolution
  - 📊 The Great Data Leakage Audit
  - 🏗️ Architecture & The "Shadow Run"
  - 🎛️ The Evidence Pack Dashboard
  - 🔧 Technology Stack Evolution
  - 💡 Key Insights Learned
  - 🚀 Next Steps

Tanzania Climate Prediction Part 2: Bridging the Valley of Death from Prototype to Production MLOps

By Walter Mnyani

📌 Project Evolution

In my previous article, I explored the foundational steps of developing a machine learning application to predict temperature patterns across Tanzania. While that initial Capstone project successfully demonstrated the predictive power of Random Forest and Ridge Regression on historical GHCN data, it ended where roughly 90% of data science projects do: as a proof-of-concept Jupyter notebook and a local Streamlit app.

However, the devastating real-world impacts of climate variability in Tanzania demand more than static prototypes. To truly support farmers, particularly in the agricultural heartland of the Kilombero Basin, we had to shift our focus from **historical temperature analysis** to **actionable rainfall prediction**, the lifeblood of crop yields. Furthermore, we needed a system robust enough that agricultural reinsurers would trust it to trigger automated, parametric insurance payouts.

This is the story of Phase 2: transforming a local prototype into a secure, automated, and mathematically verified MLOps pipeline.

📊 The Great Data Leakage Audit

When shifting our focus to rainfall prediction (precipitation mm) for parametric crop insurance, our initial models showed an astonishingly high R^2 accuracy (over 98%). However, in production machine learning, near-perfect accuracy is almost always a red flag.

Through a rigorous, automated audit of our feature engineering pipeline, we discovered a case of "silent data leakage." Our model was inadvertently training on 121 features (like `consecutive_dry_days` and `cumulative_excess_7day`) that were mathematically derived from the exact rainfall target we were trying to predict. In a real-world scenario, you wouldn't know those values until the rain had already fallen.

The Fix: We built a dedicated `data_leakage_prevention.py` module to aggressively prune any target-derived logic from the environment. We reduced our inputs from 279 candidate features down to a strictly clean schema of 83 historical and satellite features.

The result was an **honest, cross-validated XGBoost R^2 score of 86.7%**. This brutal honesty is exactly what reinsurers demand before underwriting risk.

🏗️ Architecture & The "Shadow Run"

In parametric insurance, payouts are triggered automatically when environmental data crosses a pre-defined threshold (e.g., severe drought). But how do you convince an underwriter that your AI won't trigger a million-dollar payout by mistake? You have to prove "zero look-ahead bias."

To generate this proof, we designed a **90-Day Shadow Run**.

Rather than deploying immediately with real money on the line, our pipeline is now running autonomously in a Dockerized environment on a remote server. Every single day at exactly 6:00 AM EAT (03:00 UTC), the system wakes up and executes the following sequence:

1. **Multi-Source Ingestion:** Automatically fetches the previous day's fresh atmospheric and satellite data from 5 completely distinct APIs: CHIRPS

(precipitation), NASA POWER, ERA5 (climate reanalysis), NDVI (vegetation), and Ocean Indices (ENSO/IOD phases).
2. **Model Inference:** Loads the version-controlled XGBoost model (trained strictly on historical data with a 12-month temporal gap) and generates daily forecasts for the Kilombero Basin pilot.
3. **Immutable Logging:** Saves the prediction directly to a secure `ForecastLog` database with a cryptographic timestamp.
4. **Operations Alert:** Fires an enriched Slack notification to our operations team summarizing the day's data quality, ingestion success rate, and prediction confidence.

🎛️ The Evidence Pack Dashboard

To support the Shadow Run, the user interface underwent a massive upgrade. We moved from Streamlit to a full-stack architecture using a FastAPI Python backend and a React/TypeScript frontend.

Instead of just showing historical trends, the new **Evidence Pack Dashboard** acts as an ongoing auditor. As time passes and the actual weather occurs, the backend automatically compares the 6 AM forecasts against ground truth observations. The React dashboard dynamically calculates and displays:

- **Brier Scores:** To prove the reliability of the model's probabilities.
- **Expected Calibration Error:** To show whether an "80% chance of flood" actually happens 80% of the time.
- **Confusion Matrix:** Exactly tracking False Positives (false alarms) and False Negatives (missed triggers).

At the click of a button, stakeholders can download a consolidated zip file containing all logs, metrics, and compliance attestation—the ultimate "Evidence Pack" for reinsurers.

🔧 Technology Stack Evolution

Our stack matured significantly to handle production workloads:

- **Core Backend:** Python, FastAPI, SQLAlchemy, APScheduler
- **Frontend:** React, TypeScript, Material-UI (Vite)

- **Data Ingestion:** `ecmwf-datastores-client`, Google Earth Engine API
- **Modeling:** XGBoost, Scikit-learn, Pandas
- **Infrastructure:** Docker & Docker Compose, PostgreSQL

💡 Key Insights Learned

1. **Data Freshness Trumps Model Complexity:** Running predictions at 6 AM EAT perfectly aligns with the overnight global refresh cycle of data providers like NASA and the European Space Agency. An intricate Deep Learning model is useless if the API data it relies on is 48 hours stale.
2. **Alert Fatigue is Real:** We designed our Slack integration with distinct severity channels (`#climate-pipeline-daily` vs `#climate-pipeline-alerts`) to ensure that when a critical database timeout occurs, it isn't buried under dozens of routine success messages.
3. **The Importance of Phase-Based Architectures:** We didn't just build a raw prediction model; we aligned the model outputs with the actual biological phases of rice growth in Kilombero (e.g., the Flowering phase gets a heavier payout weighting) to ensure the tech serves agricultural reality.

🚀 Next Steps

The system is now deployed. Over the next 90 days, it will run untouched, silently recording its predictions against the actual Tanzanian weather.

If the Brier Score remains below 0.25 and the basis risk holds steady, the pipeline will graduate from Shadow Run to live Commercial Pilot, unlocking real financial resilience for 1,000 rice farmers.

I look forward to sharing Part 3 of this series in June 2026: **The Results of the 90-Day Shadow Run.**